

Introdução à Programação em Python

Módulo 1: Fundamentos de Programação

O que é Programação e por que aprender?

Programação é a forma de dar instruções para o computador executar uma tarefa.

Por que aprender a programar?

Pensando na área profissional, aprender a programar é útil pois:

Alta demanda

Há uma alta demanda de profissionais no mercado de tecnologia.

Flexibilidade

Você pode trabalhar de qualquer lugar do mundo.

Remuneração

Uma das áreas mais bem remuneradas atualmente.

Introdução a Computadores

Um computador é uma máquina que recebe, processa e armazena informações.

Tudo no computador, desde as letras até cada pixel na tela é representado na forma de 0's e 1's, chamamos isso de binário.

Estrutura básica de um computador

- **CPU: Unidade Central de Processamento** - Responsável por gerenciar todas as funções do sistema
- **Memória Principal (RAM):** Guarda as informações que a CPU está usando agora
- **Memória Secundária:** Armazena seus programas e arquivos
- **Entrada e Saída:** São os periféricos como mouse, teclado, tela, impressora, etc.

Software vs Hardware

Hardware

Hardware é a parte física do computador, é a máquina em si, os circuitos eletrônicos.

Exemplos: processador, memória RAM, disco rígido, placa-mãe, monitor, teclado, mouse.

Software

Software são os programas e instruções que fazem o hardware funcionar.

Exemplos: sistema operacional, navegadores, editores de texto, jogos, aplicativos.

Algoritmos e Pseudocódigo

Um algoritmo é uma sequência de passos que visam atingir um objetivo bem definido.

Um algoritmo correto deve possuir três qualidades:

01

Instruções realizáveis

Cada passo do algoritmo deve ser uma instrução que possa ser realizada.

02

Ordem precisa

A ordem dos passos deve ser precisamente determinada.

03

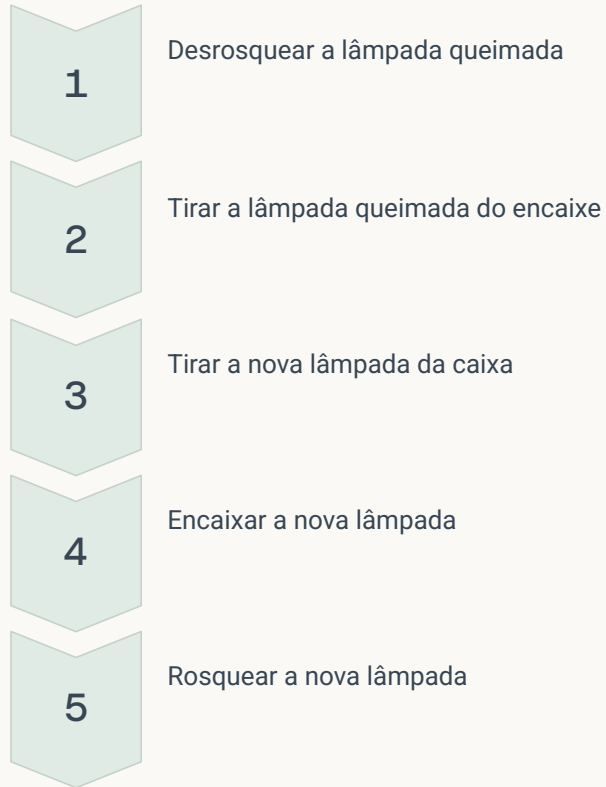
Finalização

O Algoritmo deve ter fim.

Devemos refinar um algoritmo, ou seja, melhorá-lo, até que as instruções cheguem o mais próximo possível de uma linguagem de programação.

Exemplo de Algoritmo

Algoritmo para trocar uma lâmpada



O que é Pseudocódigo?

Pseudocódigo é uma forma de escrever o passo a passo de um programa (um algoritmo), mas usando uma linguagem simples e natural (como o português) em vez de uma linguagem de programação de verdade.

Exemplo de Pseudocódigo

Pseudocódigo para somar 2 números

1

Guardar o número 1 na variável x

2

Guardar o número 2 na variável y

3

Calcular $x + y$ e guardar em z

4

Imprimir na tela o valor z

Variáveis e Tipos de Dados

Variáveis são onde guardamos os dados na memória do computador. Ela representa um espaço na memória principal que armazenaremos o dado.

Os principais tipos de dados que temos em Python são:

Inteiros (int)

1, 2, 3 ...

Float (float)

1.1, 4.2, 3.1415 ...

Booleanos (bool)

True or False

String (str)

"Isso é uma string", "a"

Como declarar uma variável em Python?

```
nome_da_variavel = valor
```


Regras para Variáveis

Regras importantes:

- Os nomes das variáveis **NÃO podem iniciar com números**
- Ela pode conter caracteres, _ e números
- Geralmente constantes escrevemos em maiúsculo
- Python é case sensitive, ou seja, NUM1 e num1 são variáveis diferentes

Como converter dados?

```
# Converter para float  
num_float = float( 145)
```

```
# Converter para inteiro  
num_int = int ("1")
```

```
# Converter para string  
string = str (4533)
```

Use `type(valor)` ou `type(nome_variavel)` para ver o tipo de um valor ou de uma variável.

- ❏ **Escopo de uma variável** define onde e quando uma variável pode ser utilizada. Em Python o escopo é definido pela indentação (afastamento do texto em relação a margem). Uma variável só pode ser acessada se estiver com a mesma indentação ou se estiver com uma indentação que esteja dentro da atual.

Operadores em Python

Temos 3 tipos de operadores em Python:

Aritméticos

```
# Adição
res = 1 + 3

# Subtração
res = 1 - 3

# Multiplicação
res = 2 * 3

# Divisão
res = 5 / 2 # 2.5

# Divisão inteira
res = 5 // 2 # 2

# Resto ou módulo
res = 5 % 2 # 1

# Potenciação
res = 5 ** 2 # 25
```

Precedência: Parênteses > ** > * > / > // > % > + > -

Uma variável pode armazenar expressões.

Relacionais

```
# Igualdade
res = 1 == 1

# Desigualdade
res = 3 != 3

# Maior e maior igual
res = 5 > 3
res = 45 >= 654

# Menor e menor igual
res = 6 < 7
res = 435 <= 435
```

Lógicos

São True ou False

- **AND (E):** Para ser True as duas condições tem que ser verdadeiras
- **OR (OU):** Para ser True uma das duas condições tem que ser verdadeiras
- **NOT (NÃO):** Inverte o valor, então se for True vira False e vice-versa

Precedência: NOT > AND > OR

Entrada e Saída

Input (Entrada)

Para o usuário inserir os dados usamos o `input`. Ele lê a entrada do teclado até apertar ENTER e lê como string. Ele pode ter um texto dentro do parênteses que será mostrado na tela antes da leitura.

Exemplo:

```
nome = input("Digite seu nome: ")
```

Print (Saída)

Para imprimir (mostrar) dados na tela usamos o `print`. Ele pode receber uma variável ou um valor.

Exemplo:

```
print("Olá, mundo!")  
print(nome)
```

Estruturas Condicionais

As vezes queremos executar um código dependendo de uma determinada condição. Para isso usamos as estruturas condicionais.

- Uma condição é uma expressão lógica, ou seja, o resultado da expressão é um valor booleano.
- Pode-se usar condicionais dentro de condicionais (dizemos que estão aninhadas).

Estrutura If-Else

```
if condição:  
    # código executado se condição for True  
else:  
    # código executado se condição for False
```

Elif

```
if condição1:  
    # código executado se condição1 for True  
elif condição2:  
    # código executado se condição2 for True  
else:  
    # código executado se nenhuma condição for True
```

Qual a diferença de usar um elif e um outro if? O elif só é verificado se a condição anterior for falsa, enquanto um novo if é sempre verificado independentemente.

Estruturas de Repetição

As vezes queremos repetir um trecho de código enquanto uma determinada condição é verdadeira. Para isso usamos as estruturas de repetição.

Loop For:

O Loop For repete um trecho de código por um número pré-determinado de vezes.

Em Python o for é usado para iterar ("percorrer") sobre alguma estrutura, diferente de outras linguagens. Assim, em Python, se queremos usar o for para executar um trecho de código 5 vezes, por exemplo, usamos uma função chamada `range()` que irá gerar uma estrutura para o for percorrer.

```
for i in range(5):  
    print(i) # Imprime 0, 1, 2, 3, 4
```

Com o range podemos especificar também da onde ele começa e o passo dele (de quantos em quantos números vamos criar). A estrutura é `range(início, fim, passo)`. Veja o exemplo:

`range(4, 10, 2)` → 4, 6, 8

Repare então que com o for podemos percorrer qualquer estrutura iterável, como: listas, string, dicionários, entre outras que veremos mais para frente.

Loop While

O Loop While repete um trecho de código enquanto uma determinada condição for verdadeira.

```
i = 0
while i < 5:
    print(i)
    i += 1 # Incrementa i em 1
```

Podemos usar esse formato `+=` para outros operadores como `-=`, `*=`, `/=`, entre outros.

Esse código irá imprimir 0, 1, 2, 3, 4 assim como o For do exemplo anterior.

Cuidados importantes:

- Tome cuidado ao usar o while pois se a sua condição não for atualizada a cada repetição você terá um loop infinito.
- Se seu objetivo é um loop infinito, podemos criar um loop infinito com uma condição que seja sempre verdade ou até mesmo usando o próprio `True` no lugar da condição.